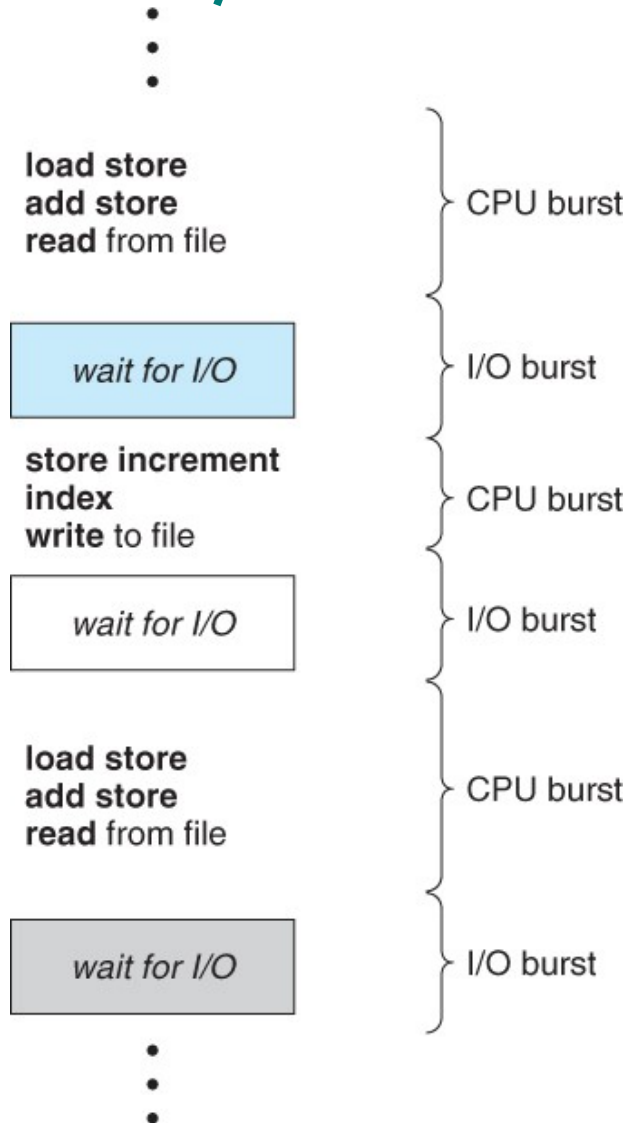


► CPU Scheduling

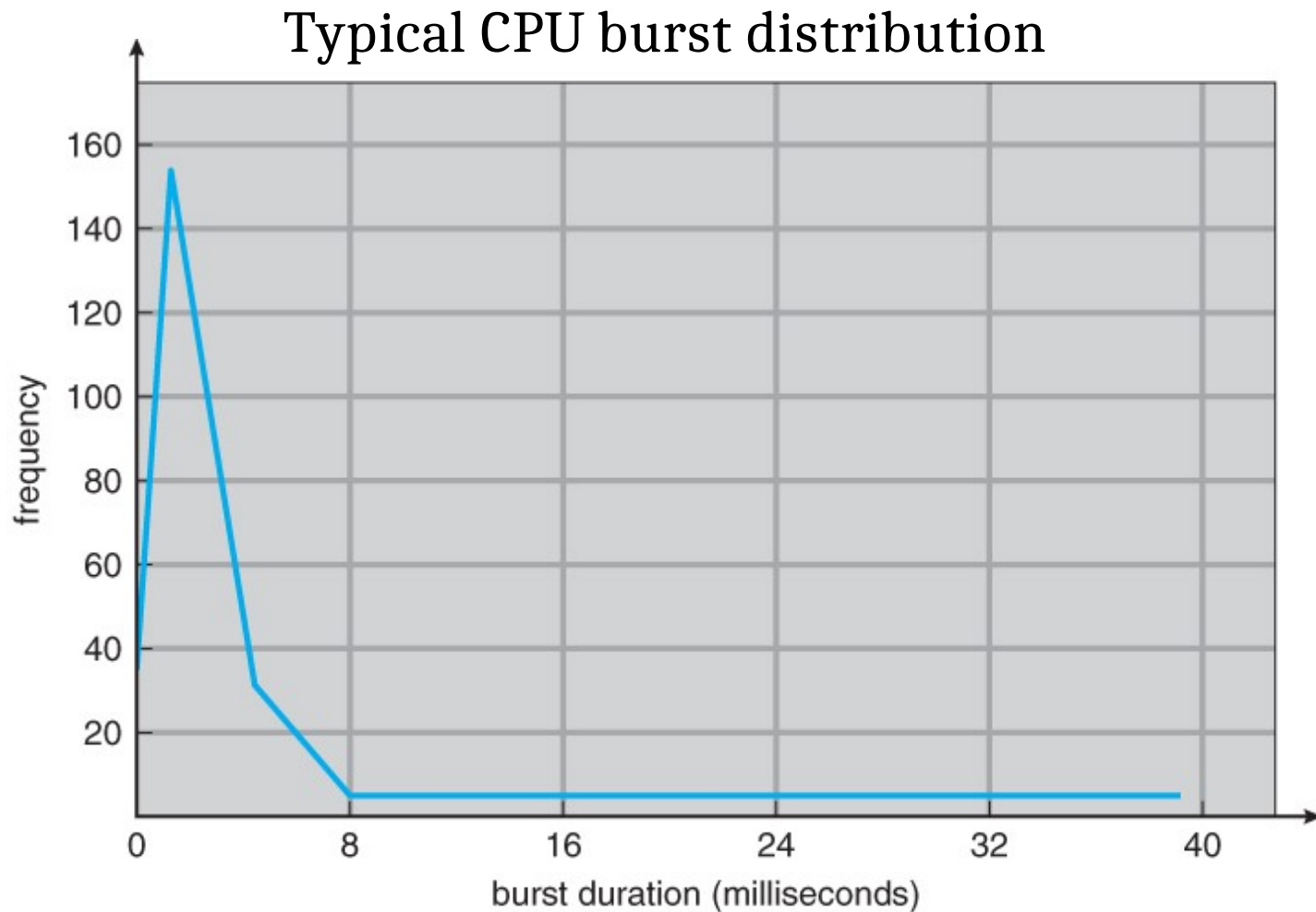
Dr. Manmath N. Sahoo
Dept. of CSE, NIT Rourkela

CPU-I/O Burst Cycle



- ▶ CPU-I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait.
- ▶ CPU burst is length of time process needs to use CPU before it next makes a I/O request.
- ▶ I/O burst is the length of time process spends waiting for I/O to complete.

Histogram of CPU-burst Times



CPU Scheduler

- Allocates CPU to one of processes that are ready to execute (in ready queue)
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state (e.g. I/O request)
 2. Terminates
 3. Switches from waiting to ready (e.g. I/O completion)
 4. Switches from running to ready state (e.g. priority based interruption)

Dispatcher

- **Dispatcher** gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode (if CPU was executing the previous process in kernel mode and preempted)
 - jumping to the proper location in the user program to restart that program (i.e., last action is to set program counter)
- **Dispatch latency** – time it takes for the dispatcher to switch between processes and start new one running

Scheduling Criteria

- **CPU utilization:** i.e. CPU usage – want to maximize
- **Throughput:** number of processes that complete their execution per time unit – want to maximize
- **Turnaround time:** amount of time to execute a particular process – want to minimize
- **Waiting time:** amount of time a process has been waiting in the ready queue – want to minimize
- **Response time:** amount of time it takes from when a job was submitted until it initiates its first response (output) – want to minimize

First-Come, First-Served Scheduling

- Schedule: order of arrival of process in ready queue

Example:	<u>Process</u>	<u>Burst Time</u>
	P_1	24
	P_2	3
	P_3	3

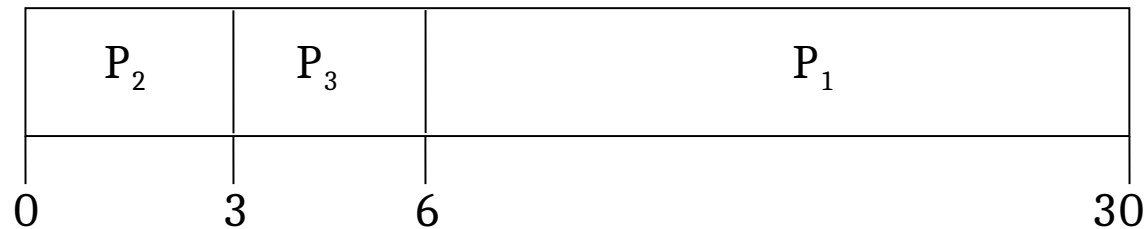
- Suppose that the processes arrive in the order: P_1, P_2, P_3 . The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

FCFS Scheduling

- Suppose that the processes arrive in the order P_2, P_3, P_1 .
- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$

FCFS Scheduling

- + Simple to implement by FIFO queue.
- + Code is easy to write and understand
- Average waiting time is not minimal
- Not suitable for time-sharing systems
- Convoy effect: short I/O intensive processes behind long CPU intensive process
- **NOTE: FCFS Scheduling is non-preemptive**

- Large CPU bound process
 - C_1 (1200ms): 1000ms computation + 200ms I/O
 - **200cpu** – 50io – **200cpu** – 50io – **200cpu** – 50io – **200cpu** – 50io – **200cpu**
- Small I/O bound processes
 - $I_1, I_2, \dots, I_{10}$
 - (8.5ms): 2.5ms computation + 8ms I/O
 - 0.5cpu – **2io** – 0.5cpu – **2io** – 0.5cpu – **2io** – 0.5cpu – **2io** – 0.5cpu
- May lead to less CPU utilization and I/O utilization → Convoy effect

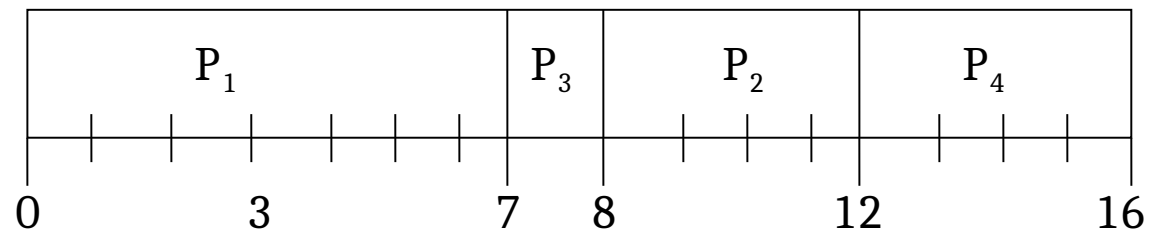
Shortest-Job-First (SJF) Scheduling

- Schedule: order of the CPU burst of the processes.
- Two schemes:
 - Non-Preemptive SJF / Shortest Job Next (SJN)
 - once CPU given to the process it cannot be preempted until completes its CPU burst.
 - Preemptive SJF / Shortest Remaining Time First (SRTF)
 - if a new process arrives with CPU burst length less than remaining time of current executing process, preempt.

Non-Preemptive SJF (SJN)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

- Gantt Chart:

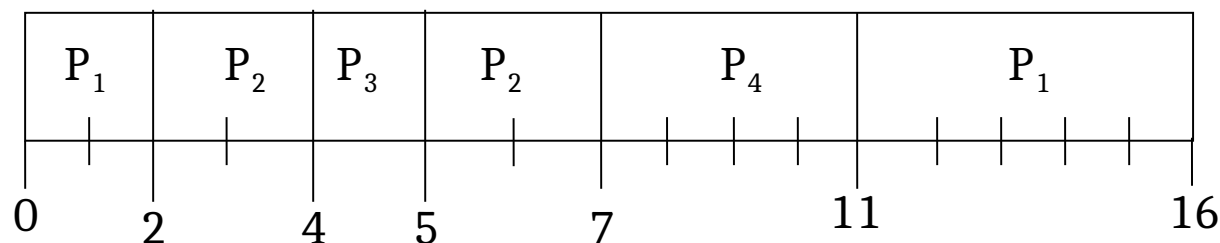


- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Preemptive SJF (SRTF)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

- Gantt Chart:



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Example

<u>Process</u>		<u>Arrival Time</u>	<u>Burst Time</u>
<i>P1</i>	0	16	
<i>P2</i>	2	8	
<i>P3</i>	4	12	
<i>P4</i>	6	2	
<i>P5</i>	8	4	

- Draw Gantt chart and find the average waiting time in each of the following cases:
 - SRTF Scheduling
 - SJN Scheduling
 - FCFS Scheduling

Answer to Example

<u>Process</u> <u>Time</u>	<u>Arrival Time</u>	<u>Burst</u>
P1	0	16
P2	2	8
P3	4	12
P4	6	2
P5	8	4

■ SRTF



■ Average waiting time = $(26+6+12+0+0)/5$
 $= 8.8$

Answer to Example

<u>Process</u> <u>Time</u>	<u>Arrival Time</u>	<u>Burst</u>
<i>P1</i>	0	16
<i>P2</i>	2	8
<i>P3</i>	4	12
<i>P4</i>	6	2
<i>P5</i>	8	4

■ SJN

P1		P4		P5		P2		P3	
0	16	18	22	30	42				

■ Average waiting time = $(0+20+26+10+10)/5$
 $= 13.2$

Answer to Example

<u>Process</u> <u>Time</u>	<u>Arrival Time</u>	<u>Burst</u>
<i>P1</i>	0	16
<i>P2</i>	2	8
<i>P3</i>	4	12
<i>P4</i>	6	2
<i>P5</i>	8	4

■ FCFS



■ Average waiting time = $(0+14+20+30+30)/5$
 $= 18.8$

Answer to Example

Scheduling Algorithm	Average Waiting Time
FCFS	18.8
SJN	13.2
SRTF	8.8

Shortest Job First

- + SRTF is Optimal: Minimum Average waiting time.
- Longer processes may be starved.
- Difficult to know the CPU burst of processes before their execution

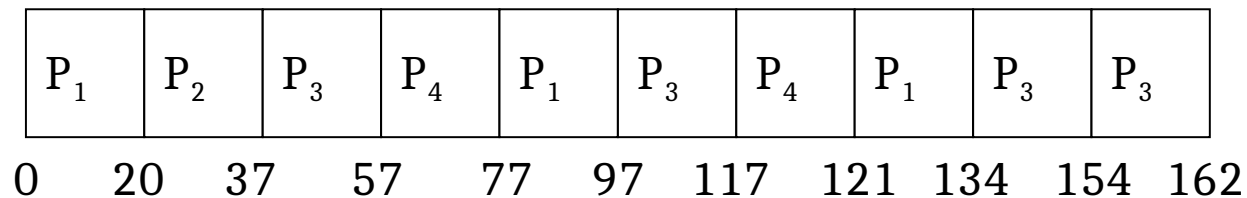
Round Robin (RR) Scheduling

- Each process gets a **time quantum**, usually 10-100 milliseconds.
- After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time.
- No process waits more than $(n-1)q$ time units.

Example: RR with $q=20$

<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*.

Round Robin (RR) Scheduling

- FCFS + Preemption = RR
- Performance
 - q large $\Rightarrow$ behaves similar to FCFS
 - q small $\Rightarrow$ more context switch overhead

Modified RR Scheduling

- Based on variable time quantum
- A preempted process, on its next turn will use an increased time quantum
- RR Scheduling
 - P_1 : 300ms, $q=20$ ms, time per context switch=2ms
 - #of preemptions due to this process = 14
 - Preemption overhead = 28ms

Modified RR Scheduling

- Modified RR Scheduling – q is doubled in next turn.
 - P_1 : 300ms, $q=20$ ms (initial)
 - #of preemptions due to this process = 3.
($20+40+80+160=300$)
 - Preemption overhead = 6ms

Priority Scheduling

- A priority number (integer) is associated with each process and the CPU is allocated to the process with the highest priority
- Some schemes use smallest integer $\equiv$ highest priority.
- SJF is effectively priority scheduling where priority is defined on next CPU burst time.
- FCFS is effectively priority scheduling where priority is defined on arrival time.

Priority Scheduling

- Problem: **Starvation** – low priority processes may never execute.
- Solution: **Aging** – as time progresses increase the priority of the process.
 - e.g. in every 10ms increase the priority of process by 1

Highest Response Ratio Next (HRRN) Scheduling

- It gives priority to shorter jobs similar to SJN
- Implements Aging implicitly
- Non-preemptive dynamic priority based algorithm with initial priority 1
- $p = 1 + (\text{wait time} / \text{burst time})$

Highest Response Ratio Next (HRRN) Scheduling

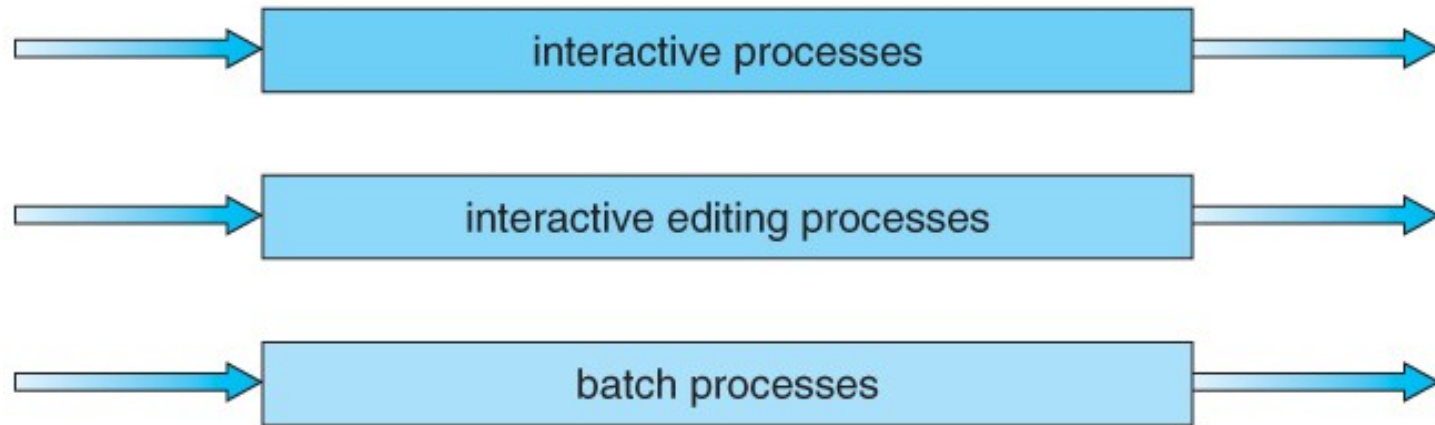
<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
<i>P1</i>	0	3
<i>P2</i>	2	6
<i>P3</i>	4	4
<i>P4</i>	6	5
<i>P5</i>	8	2



Multi level Queue Scheduling

- Ready queue is partitioned into separate queues.
- Each queue holds a particular category of processes.
- Each queue can implement whatever scheduling algorithm is most appropriate for that type of processes.

Multi level Queue Scheduling



Multilevel Queue Scheduling

- Scheduling must be done between the queues.
 - Priority scheduling; But possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes;

Multilevel Queue Scheduling

■ Objectives:

- To optimize turnaround time.
- To minimize response time (to make a system feel responsive to interactive users).

THE CRUX:

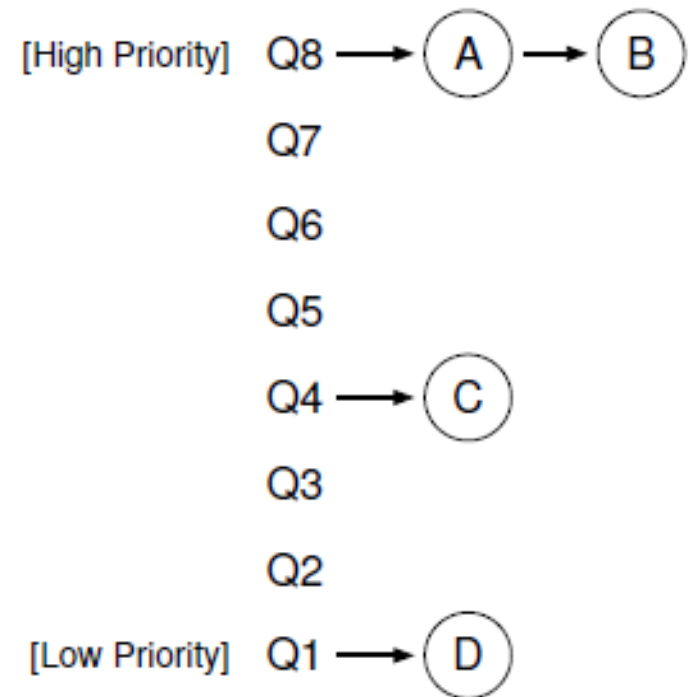
HOW TO SCHEDULE WITHOUT PERFECT KNOWLEDGE?

How can we design a scheduler that both minimizes response time for interactive jobs while also minimizing turnaround time without *a priori* knowledge of job length and job behaviour?

Multilevel Feedback Queue Scheduling -Basic Rules

- >1 number of queues, each assigned a different priority level
- Processes are executed from top priority queue to low priority queue.

- Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR/SJN/any other.



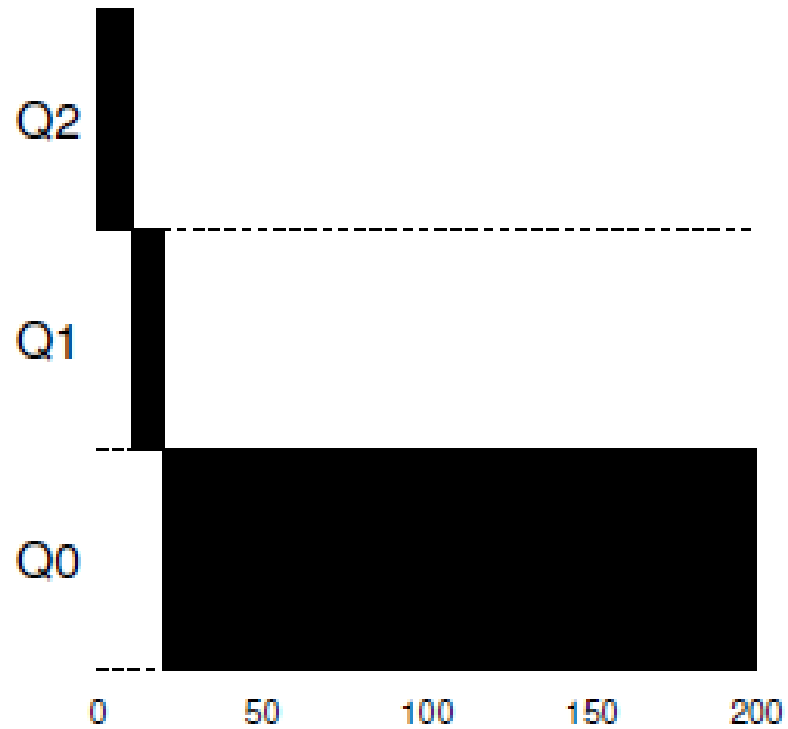
MLFQ: Basic Rules

- MLFQ varies the priority (dynamic) of a job based on its observed behavior.
- If a process repeatedly relinquishes the CPU, MLFQ will keep its priority high.
- If a job uses the CPU intensively for long periods of time, MLFQ will reduce its priority.

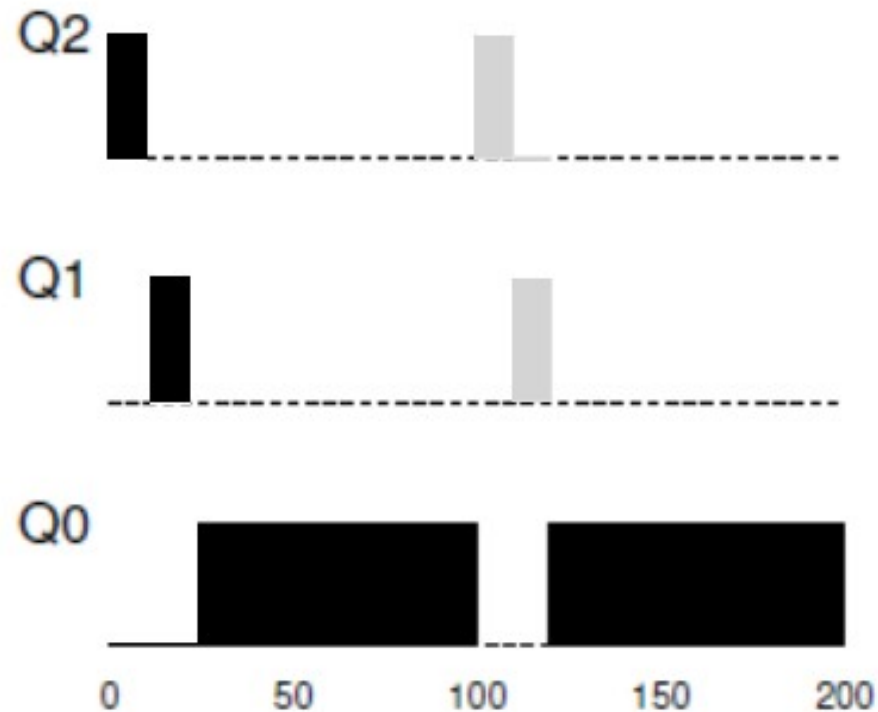
MLFQ Priority Adjustment: Attempt# 1

- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4a:** If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue).
- **Rule 4b:** If a job gives up the CPU before the time slice is up, it stays at the same priority level.

Example 1: A Single Long-Running Job

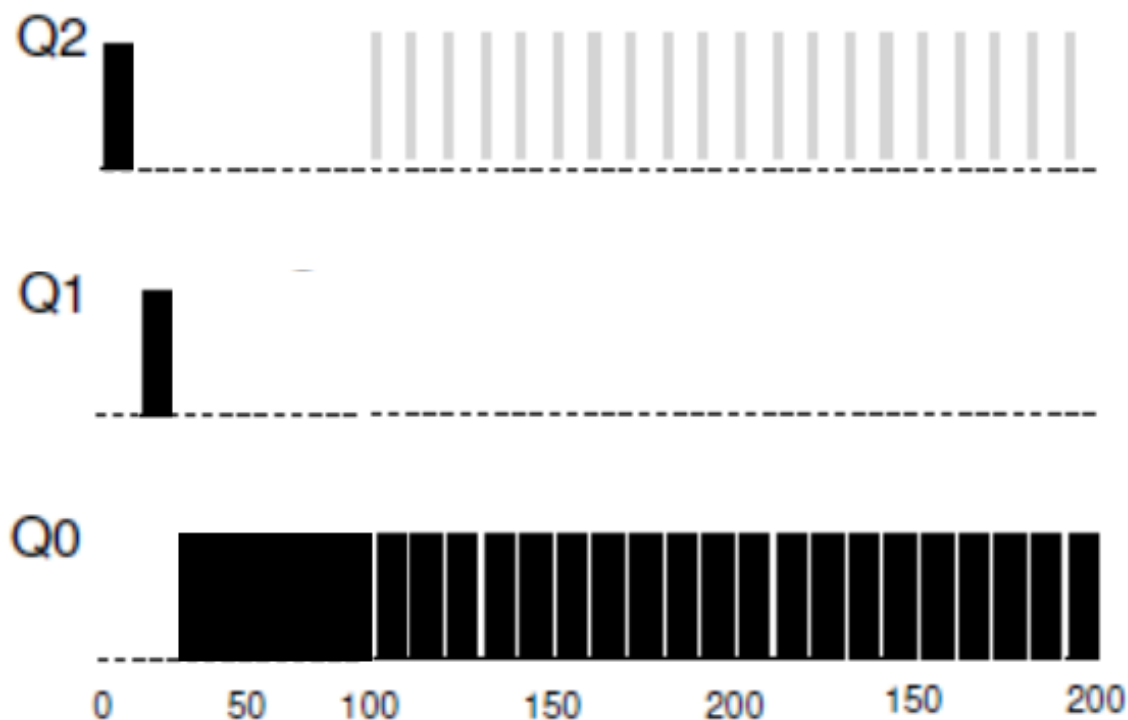


Example 2: Along Came A Short Job (20ms) at 100ms



Example 3: What About I/O?

- An interactive job B needs the CPU only for 1ms before performing an I/O of 9ms.



Problems With Current MLFQ

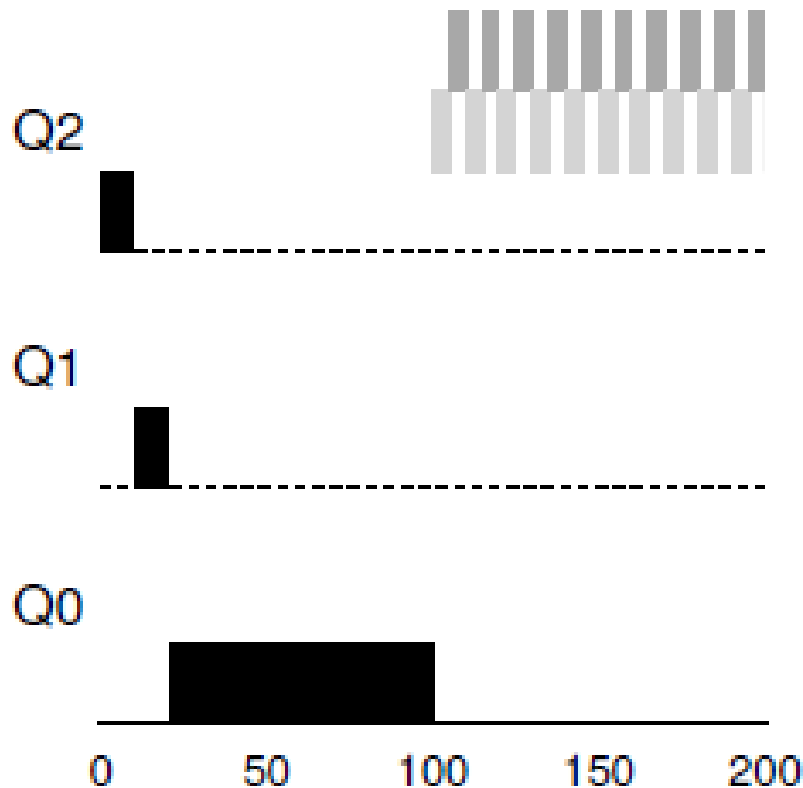
- Starvation to long jobs.
- A smart user could rewrite their program to **game the scheduler.**
- A process may change its behavior over time
 - from CPU bound to a phase of interactivity.

Attempt #2: The Priority Boost

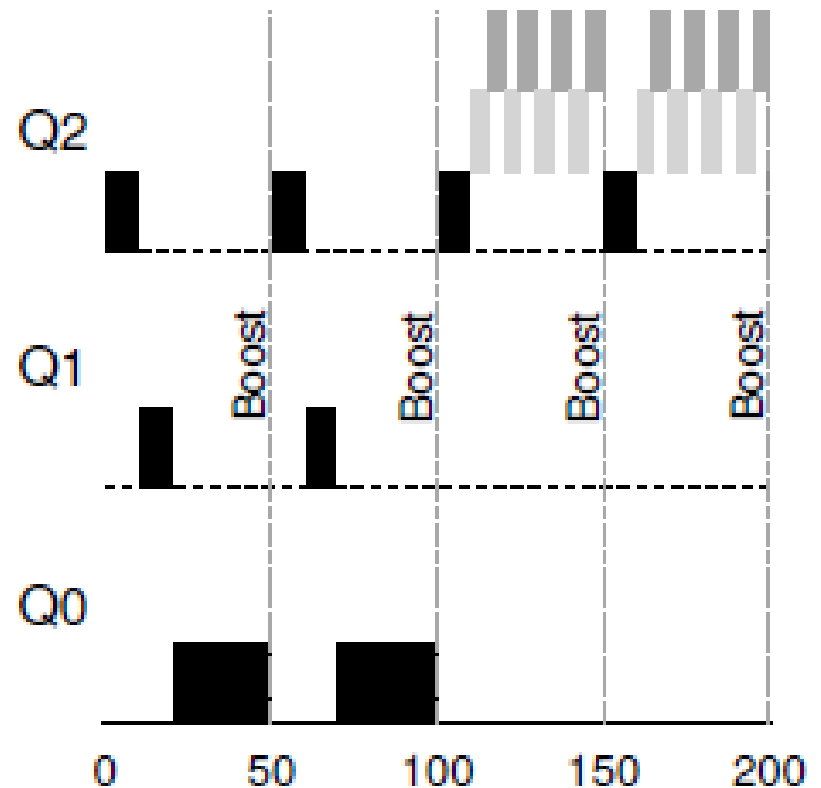
- guarantees that CPU-bound jobs will make some progress (even if it is not much).
- Rule 5: After some time period S , move all the jobs in the system to the topmost queue.

Attempt #2: The Priority Boost

- interactive jobs need CPU only for 5ms before performing an I/O of 5ms.



(a) Without Priority Boost



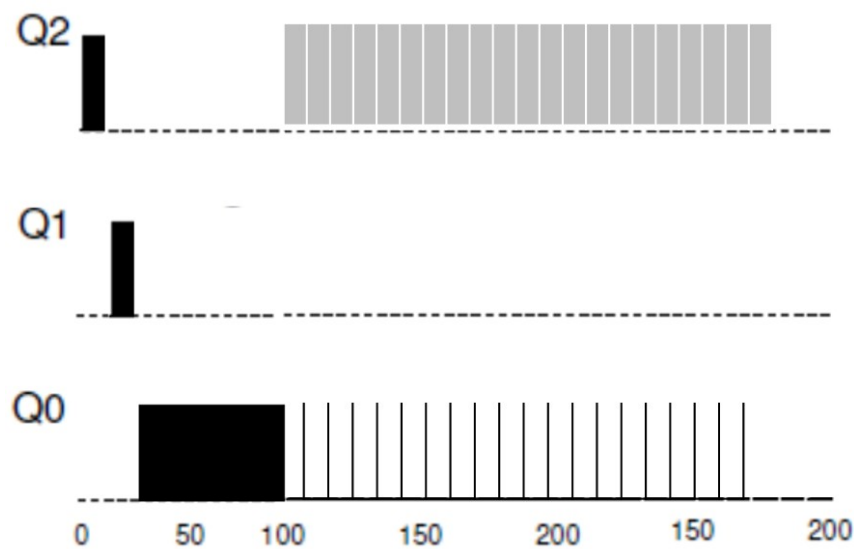
(b) With Priority Boost in every 50ms

Attempt #3: Better Accounting

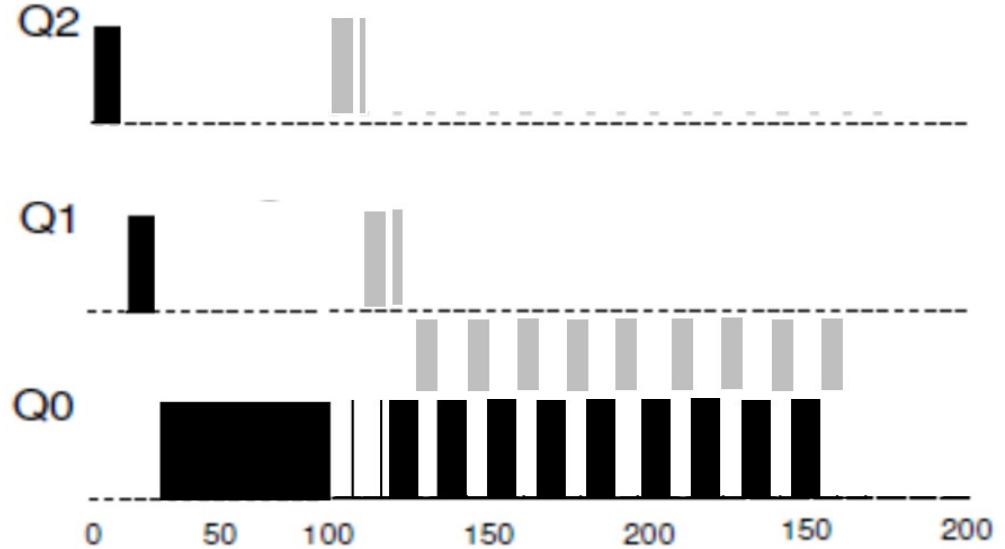
- To prevent gaming of the scheduler, Rules 4a and 4b are rewritten to the following single rule:

- **Rule 4:** Once a job uses up its time quantum at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).

Effect of anti-Gaming rule



(a) Without anti-Gaming rule



(b) With anti-Gaming rule